Técnico Superior en Desarrollo de Aplicaciones Web

Despliegue de Aplicaciones Web

Practica SVC-GIT

Lee bien todo lo que se indica en este enunciado y procura respetarlo.

Si tienes alguna duda o crees que hay alguna incoherencia, avisa al profesor para que te la solucione.

1.Introducción

Este es un ejercicio práctico que consta de dos documentos:

- Documento con el enunciado (el que estás leyendo ahora mismo)
- Documento con la plantilla para las respuestas.

En el enunciado vas a encontrar una serie de tareas que debes realizar en tu PC. También vas a encontrar varias preguntas que deberás responder en la plantilla de respuestas.

Al final del ejercicio debes tener tu plantilla de respuestas rellenas y un directorio con archivos en tu PC.

2. Qué se entrega

Cada alumno debe entregar :

1. El documento de respuestas **al profesor**
2. El directorio con los archivos que obtiene tras completar las tareas del enunciado comprimido **en la plataforma**

El profesor corrige lo que se entrega, así que **ASEGURATE BIEN DE QUÉ ENTREGAS.**

3. Plagio

Siguiendo los estatutos del centro, se considerará falta grave cuando se detecte que el trabajo de un alumno/a muestre síntomas evidentes de plagio en la resolución de algunas de las tareas indicados más adelante, ya sea por contenido proporcionado por otro alumno/a, por contenido copiado literalmente de internet o por contenido generado por una IA. En ese caso, la calificación de esta practica será de 0 y se valorará si el alumno suspende toda la parte práctica del trimestre.

4. Objetivos didácticos

En esta práctica vas a trabajar con un repositorio Git local. No se trata de memorizar comandos, sino de comprender cómo funciona Git y cómo evoluciona el historial de un proyecto.

Al finalizar, deberás demostrar que sabes:

- Crear un repositorio y guardar cambios mediante commits.
- Configurar un archivo `.gitignore` para excluir archivos del seguimiento de Git.
- Crear una rama y trabajar en ella de forma independiente.
- Comprender cómo se desplazan los punteros de las ramas y `HEAD`.
- Fusionar dos ramas y representar gráficamente el resultado.

Lo importante es que sepas explicar qué ha ocurrido y por qué.

5. Indicaciones importantes

- No elimines el repositorio ni vuelvas a inicializarlo durante la práctica.
- No crees commits distintos de los solicitados.
- Comprueba el estado del repositorio siempre que tengas dudas.
- Los dibujos deben representar el historial real que has construido, no un esquema genérico.
- En cada dibujo, identifica claramente qué commit señala cada rama y dónde se encuentra `HEAD`.
- Se valorará más una explicación razonada que una respuesta memorizada.
- Si cometes un error, intenta comprender qué ha ocurrido antes de continuar.

ENUNCIADO

Fase 1. Crear el repositorio y el primer commit (15 minutos)

Paso 1. Crea un repositorio local llamado `PracticaGit`.

Paso 2. Dentro del repositorio, crea los siguientes archivos:

- `README.md`, con tu nombre completo.
- `index.html`, con este contenido:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi proyecto</title>
</head>
<body>
    <h1>Mi primera página</h1>
</body>
</html>
```

Paso 3: Pasa todos los archivo al área de preparación y crea un commit con el mensaje: `Inicio`

Paso 4: Crea el archivo `estilos.css` y añade este contenido:

```
body{
    background-color: navy;
}
h1{
    color: #FFF;
}
```

Paso 5. Crea un archivo llamado `.gitignore` que ignore los archivos cuya extensión sea `.tmp`

Paso 6. Crea un archivo llamado `borrador.tmp` y escribe: el nombre de uno de los compañeros que tienes a tu lado (si no tienes a nadie, pon tu nombre).

Paso 7. Añade al área de preparación los archivos que no estén en el commit anterior y crea un nuevo commit con el mensaje: `Estilos`

Pregunta 1. Comprueba el estado del repositorio y responde:

a) ¿se ha guardado el archivo `borrador.tmp` en el commit? ¿Por qué?

b) ¿Se ha borrado `borrador.tmp` del disco o sigue existiendo?

c) Si se elimina la regla que has puesto en el `.gitignore` ¿Se añadiría automáticamente el archivo al repositorio?

Fase 2. Crear dos líneas de desarrollo (20 minutos)

A partir de ahora, el proyecto tendrá una rama principal y una rama secundaria.

Paso 8. Crea una rama llamada `diseno` a partir del estado actual del proyecto.

Paso 9. Vuelve a la rama `main/master`.

En el archivo `README.md`, añade una segunda línea con este texto:

```
Cambio realizado en la rama principal
```

Guarda el archivo y crea un commit con el mensaje: `Actualiza README`

Paso 10. Cambia a la rama `diseno`.

En el archivo `index.html`, modifica el `<head>` para enlazar el archivo CSS:

```
<link rel="stylesheet" href="estilos.css">
```

Crea un commit con el mensaje: `Cambia diseño`

Pregunta 2. Dibujo obligatorio número 1.

Dibuja el historial completo del repositorio en este momento.

En el dibujo deben aparecer:

- Los commits que existen.
- Las conexiones entre ellos.
- Los punteros de todas las ramas que existen.
- La posición de `HEAD` en este momento.

Importante: no basta con dibujar dos líneas y escribir sus nombres. Los punteros deben señalar al commit correcto.

Pregunta 3. Responde razonadamente:

a) ¿Qué commits componen la rama `diseño`?

b) Si cambias a `main/master`, ¿Tiene aplicado el css el archivo index.html? Explica tu respuesta.

Fase 3. Fusionar las ramas (15 minutos)

Paso 11. Fusiona la rama `diseno` dentro de `main/master`. Es decir, la rama `main/master` debe recibir los cambios realizados en `diseno`.

Si Git solicita confirmar el mensaje de la fusión, utiliza: `Fusiona diseno`

No modifiques los archivos durante esta operación ni crees commits adicionales.

Pregunta 4. Dibujo obligatorio número 2.

Dibuja el historial completo del repositorio en este momento.

En el dibujo deben aparecer:

- Los commits que existen.
- Las conexiones entre ellos.
- Los punteros de todas las ramas que existen.
- La posición de `HEAD` en este momento.

Importante: no basta con dibujar dos líneas y escribir sus nombres. Los punteros deben señalar al commit correcto.

Pregunta 5. Contesta:

a) ¿La rama `diseno` ha desaparecido después de fusionarla? ¿Por qué?

b) ¿Ha habido algún conflicto cuando has fusionado? Explica el motivo de tu respuesta.

Fase 4. Comprobar y explicar el historial (10 minutos)

Paso 12: Edita el archivo `README.md` y añade la linea:

```
Version Final
```

Paso 13: Guarda el archivo y crea un commit con el mensaje: `Acabado`

Paso 14: Nombra ese commit con la etiqueta: `RC-Version`

Paso 15: Ejecuta:

```
git log --oneline --graph --all
```

Guarda una captura del resultado y adjuntala en la entrega.

Pregunta 6. Explica con tus propias palabras:

a) ¿Qué diferencia hay entre un commit y una rama?

b) Cuando creas una rama nueva, ¿se duplican todos los archivos del proyecto en una nueva carpeta? ¿Por qué?

c) Si ahora vuelves a `diseno` y creas otro commit, ¿se moverá también el puntero `main/master`? ¿por qué?

d) ¿Qué ventaja tiene trabajar en ramas en lugar de hacer todos los cambios directamente en `main/master`? Pon un ejemplo de todos los que dije en clase.